

计算机系统基础 — PA1

一、实验实现流程

1. 引言

在 PA 流程中，我们最终需要实现 NEMU，一个 x86 虚拟机。在实验开始之前，对于虚拟化作出一定的辨析：虚拟化的核心是通过程序模拟目标系统功能，在它上面运行的程序无法区分真实与虚拟环境。而虚拟机实际上也是一种程序，本质是“执行其他程序的程序”，NEMU 作为 x86 虚拟机，其核心在于模拟 n86（x86 子集）硬件系统，使其可以运行相应的程序。

在本次实验中，需要使用 C 语言编写代码，从而在 NEMU 里面实现一个完整的图灵机，接下来将详细讲解代码的实现流程。

2. 修复寄存器 (reg.h)

在本部分，将修复 reg.h 代码中的寄存器的实现，解决编译报错问题。从计算机硬件的角度来看，同一寄存器表示不同位时需要共享同一块内存，依照指导书要求，本实验中使用 union 来实现寄存器。修改后的主要代码如下：

```
1      typedef struct {
2          union {
3              // 数组形式访问
4              struct {
5                  union {
6                      uint32_t _32; // 32位寄存器 (EAX)
7                      uint16_t _16; // 低16位 (AX)
8                      uint8_t _8[2]; // 低8位 (AL) + 高8位 (AH)
9                  };
10             } gpr[8];
11
12             // 别名形式访问
13             struct {
14                 rtlreg_t eax, ecx, edx, ebx, esp, ebp, esi, edi;
15             };
16         };
17     };
18 }
```

```

16     };
17
18     vaddr_t eip;
19
20 } CPU_state;

```

3. 实现基础调试器 (.../debug/)

依据实验要求，需要在调试器部分实现 si、info r 与 x 的调试功能，其中 si 对应单步执行，info r 用于打印寄存器状态，x 则是对应的内存扫描。对于涉及的功能，主要在 ui.c 文件内进行实现。

下面是添加的实现如上功能的函数：

```

1  // 单步执行
2  static int cmd_si(char *args) {
3      int n = 1;
4      if (args != NULL) {
5          n = atoi(args);
6      }
7      cpu_exec(n);
8      return 0;
9  }
10
11 // 查看寄存器
12 static int cmd_info(char *args) {
13     if (args != NULL && strcmp(args, "r") == 0) {
14         printf("eax: 0x%08x\tecx: 0x%08x\tedx: 0x%08x\tebx: 0x%08x\n",
15             cpu.eax, cpu.ecx, cpu.edx, cpu.ebx);
16         printf("esp: 0x%08x\tebp: 0x%08x\tesi: 0x%08x\tedi: 0x%08x\n",
17             cpu.esp, cpu.ebp, cpu.esi, cpu.edi);
18         printf("eip: 0x%08x\n", cpu.eip);
19         return 0;
20     }
21     printf("Usage: info r\n");
22     return 0;
23 }
24
25 // x N addr: 从地址addr开始，查看N个4字节内存数据
26 static int cmd_x(char *args) {
27     // 分割N和地址

```

```

26 char *arg_n = strtok(args, " ");
27 char *arg_addr = strtok(NULL, " ");
28
29 if (arg_n == NULL || arg_addr == NULL) {
30     printf("Usage: x N addr\n");
31     return 0;
32 }
33
34 // 转换参数
35 int n = atoi(arg_n);
36 vaddr_t addr = strtoul(arg_addr, NULL, 16);
37
38 printf("Memory view at 0x%08x:\n", addr);
39 int i;
40 for (i = 0; i < n; i++) {
41     // 读取addr处4字节数据
42     uint32_t data = vaddr_read(addr, 4);
43     printf("0x%08x: 0x%08x\n", addr, data);
44     addr += 4;
45 }
46 return 0;
47 }

```

在这里，cmd_si 通过传入参数调用 cpu_exec，完成单步/多步执行；cmd_info 判断参数格式，按要求打印全部通用寄存器与指令指针的值；cmd_x 解析参数获取长度与地址，通过循环调用内存读取接口，逐地址输出 4 字节数据。

同时，为了新定义的功能可以正常使用，还需要将对应的命令注册到 cmd_table 列表里，具体代码将在后面统一给出。

4. 表达式求值

在这一部分，主要涉及 expr.c 文件的实现，我们首先要完成计算表达式的逻辑功能，然后再在 ui.c 中进行命令调用即可完成相应的功能。

expr.c 的代码主要实现了一个递归下降解析器，从加减到乘除再到括号与数值逐层递归，从而实现按照一定优先级的表达式计算。在具体实现方面，下面代码在调用时先从入口开始，递归调用分析加减法，当在加减法的解析过程之中继续递归调用乘除法的解析，以此类推，乘除法的解析继续递归调用括号与数值（包括具体数字与寄存器），最后一层计算完毕后，依照递归的栈的特性，逆序返回逐层递归的计算结果值，以实现优先括号，其次乘除，再次加减的四则运算计算规则。

在代码编写过程中，也遇到了一些问题，例如用于对比寄存器名称的 strcmp 函数

总是报错。检查代码询问 AI 后得知，是 strcmp 由于头文件的宏产生了冲突，因此此处使用字符类型（char）逐字进行比对，手动实现寄存器名称的判别。

```

1 // 递归下降解析
2 static uint32_t parse_expr(bool *success);
3 static uint32_t parse_term(bool *success);
4 static uint32_t parse_primary(bool *success);
5
6 /* 处理括号、数字、寄存器 */
7 static uint32_t parse_primary(bool *success) {
8     if (!lookahead()) { *success = false; return 0; }
9     char *t = token();
10    uint32_t val = 0;
11
12    if (t[0] == '(') {
13        free(t);
14        val = parse_expr(success);
15        if (!*success) return 0;
16        if (!lookahead()) { *success = false; return 0; }
17        t = token();
18        if (t[0] != ')') { *success = false; free(t); return 0; }
19        free(t);
20        return val;
21    }
22    else if (t[0] == '$') {
23        // 手动比较寄存器名，防止 strcmp 宏冲突
24        char *reg_name = t + 1; // 跳过 '$'
25        uint32_t val = 0;
26        bool found = false;
27
28        if (reg_name[0] == 'e' && reg_name[1] == 'a' && reg_name[2] == 'x'
29            && reg_name[3] == '\0') { val = cpu.eax; found = true; }
30        else if (reg_name[0] == 'e' && reg_name[1] == 'c' && reg_name[2]
31            == 'x' && reg_name[3] == '\0') { val = cpu.ecx; found = true;

```

```

    }
32     else if (reg_name[0] == 'e' && reg_name[1] == 's' && reg_name[2]
        == 'p' && reg_name[3] == '\0') { val = cpu.esp; found = true;
        }
33     else if (reg_name[0] == 'e' && reg_name[1] == 'b' && reg_name[2]
        == 'p' && reg_name[3] == '\0') { val = cpu.ebp; found = true;
        }
34     else if (reg_name[0] == 'e' && reg_name[1] == 's' && reg_name[2]
        == 'i' && reg_name[3] == '\0') { val = cpu.esi; found = true;
        }
35     else if (reg_name[0] == 'e' && reg_name[1] == 'd' && reg_name[2]
        == 'i' && reg_name[3] == '\0') { val = cpu.edi; found = true;
        }
36
37     free(t);
38     if (found) { *success = true; return val; }
39     else { *success = false; return 0; }
40 }
41 else if (isdigit(t[0]) || (t[0] == '0' && (t[1] == 'x' || t[1] == '
    X')))) {
42     val = strtoul(t, NULL, (t[0] == '0' && (t[1] == 'x' || t[1] == 'X
        ')) ? 16 : 10);
43     free(t);
44     *success = true;
45     return val;
46 }
47 else { *success = false; free(t); return 0; }
48 }
49
50 /* 处理乘除 */
51 static uint32_t parse_term(bool *success) {
52     uint32_t val = parse_primary(success);
53     if (!*success) return 0;
54
55     while (1) {
56         if (!lookahead()) { *success = true; return val; }
57         char *t = token();
58         if (t[0] != '*' && t[0] != '/') {
59             pos -= token_len; // 回退 token
60             free(t);

```

```

61     *success = true;
62     return val;
63 }
64 char op = t[0];
65 free(t);
66
67 uint32_t rhs = parse_primary(success);
68 if (!*success) return 0;
69 if (op == '*') val *= rhs;
70 else if (op == '/') { if (rhs == 0) { *success = false; return 0;
    } val /= rhs; }
71 }
72 }
73
74 /* 处理加减 */
75 static uint32_t parse_expr(bool *success) {
76     uint32_t val = parse_term(success);
77     if (!*success) return 0;
78
79     while (1) {
80         if (!lookahead()) { *success = true; return val; }
81         char *t = token();
82         if (t[0] != '+' && t[0] != '-') {
83             pos -= token_len; // 回退 token
84             free(t);
85             *success = true;
86             return val;
87         }
88         char op = t[0];
89         free(t);
90
91         uint32_t rhs = parse_term(success);
92         if (!*success) return 0;
93         if (op == '+') val += rhs;
94         else if (op == '-') val -= rhs;
95     }
96 }
97
98 // 框架入口
99 uint32_t expr(char *e, bool *success) {

```

```

100     expr_str = e;
101     pos = 0; // 每次调用重置解析位置
102     *success = true;
103
104     uint32_t val = parse_expr(success);
105     if (!*success) return 0;
106
107     // 检查是否解析完所有 token
108     if (lookahead()) { *success = false; return 0; }
109     return val;
110 }

```

完成了主要的逻辑后，和前面的流程一样，继续将命令配置在 ui.c 的代码中：

```

1 // p 功能(计算表达式)
2 static int cmd_p(char *args) {
3     if (args == NULL) {
4         printf("Usage: p EXPR\nExample: p 1+2*3, p $eax, p 0x100000\n");
5         return 0;
6     }
7
8     bool success;
9     // 调用框架的表达式求值函数
10    uint32_t result = expr(args, &success);
11
12    if (success) {
13        printf("0x%08x (%u)\n", result, result);
14    } else {
15        printf("Invalid expression!\n");
16    }
17    return 0;
18 }

```

5. 监视器实现

接下来，将继续进行监视点相关功能的实现。查看给出的 watchpoint.h 与 watchpoint.c 代码框架，依照指导书要求，采用监视点池来维护资源。在这里，我们使用最基础的链表作为底层结构来实现。

在 watchpoint.h 里面定义好所需的变量与函数声明后，在 watchpoint.c 内添加如下代码。依照框架，首先用静态数组构建监视点池，初始化时将所有节点串为 free_ 空闲

链表。新建监视点时从空闲链摘取节点，校验表达式并记录初始值后加入 head 已用链表；删除监视点时，则是从已用链移除节点归还空闲链，方便后续复用。check_wp 进行检查时，遍历已用链检测表达式值变化，触发则暂停模拟器；list_wp 则是直接遍历已用链表，用于展示信息。

```

1 // 新建监视点
2 WP* new_wp(char *expr_str) {
3     if (free_ == NULL) {
4         printf("Error: Too many watchpoints!\n");
5         return NULL;
6     }
7
8     // 从空闲链表取出一个
9     WP *wp = free_;
10    free_ = free_>next;
11
12    // 计算表达式初始值
13    bool success;
14    uint32_t init_val = expr(expr_str, &success);
15    if (!success) {
16        printf("Error: Invalid expression!\n");
17        // 放回空闲链表
18        wp->next = free_;
19        free_ = wp;
20        return NULL;
21    }
22
23    // 赋值
24    strncpy(wp->expr, expr_str, CMD_LEN);
25    wp->old_val = init_val;
26
27    // 插入使用链表头部
28    wp->next = head;
29    head = wp;
30
31    printf("Watchpoint %d created: %s = 0x%08x\n", wp->NO, wp->expr,
32           init_val);
33    return wp;
34 }

```

```

35 // 删除监视点
36 void free_wp(int NO) {
37     if (head == NULL) {
38         printf("No watchpoints to delete!\n");
39         return;
40     }
41
42     WP *prev = NULL, *curr = head;
43     while (curr != NULL && curr->NO != NO) {
44         prev = curr;
45         curr = curr->next;
46     }
47
48     if (curr == NULL) {
49         printf("Watchpoint %d not found!\n", NO);
50         return;
51     }
52
53     // 从使用链表移除
54     if (prev == NULL) head = curr->next;
55     else prev->next = curr->next;
56
57     // 放回空闲链表
58     curr->next = free_;
59     free_ = curr;
60
61     printf("Watchpoint %d deleted\n", NO);
62 }
63
64 // 检查监视点值是否变化
65 void check_wp(void) {
66     WP *wp = head;
67     while (wp != NULL) {
68         bool success;
69         uint32_t new_val = expr(wp->expr, &success);
70
71         if (success && new_val != wp->old_val) {
72             printf("\n[Watchpoint %d Triggered!]\n", wp->NO);
73             printf("Expr: %s\nOld value: 0x%08x\nNew value: 0x%08x\n",
74                 wp->expr, wp->old_val, new_val);

```

```

75     wp->old_val = new_val;
76     // 触发暂停, 返回命令行
77     nemu_state = NEMU_STOP;
78 }
79 wp = wp->next;
80 }
81 }
82
83 // 列出所有监视点
84 void list_wp(void) {
85     if (head == NULL) {
86         printf("No watchpoints.\n");
87         return;
88     }
89
90     printf("Num\tExpr\n");
91     WP *wp = head;
92     while (wp != NULL) {
93         printf("%d\t%s\n", wp->NO, wp->expr);
94         wp = wp->next;
95     }
96 }

```

完成上述核心功能后, 在 ui.c 添加相应的命令:

```

1 // 监视点命令 w EXPR
2 static int cmd_w(char *args) {
3     if (args == NULL) {
4         printf("Usage: w EXPR\nExample: w $eax, w 0x100000\n");
5         return 0;
6     }
7     new_wp(args);
8     return 0;
9 }
10
11 // 删除监视点 d 编号
12 static int cmd_d(char *args) {
13     if (args == NULL) {
14         printf("Usage: d WP_NUMBER\n");
15         return 0;
16     }

```

```

17     free_wp(atoi(args));
18     return 0;
19 }

```

截至目前，PA1 所需功能均已实现，整理 cmd_table[] 列表，添加以上所有功能：

```

1 cmd_table [] = {
2     { "help", "Display informations about all supported commands",
        cmd_help },
3     { "c", "Continue the execution of the program", cmd_c },
4     { "q", "Exit NEMU", cmd_q },
5     { "si", "Single step execution", cmd_si },
6     { "info", "Display register info (info r)", cmd_info },
7     { "x", "Examine memory: x N addr", cmd_x },
8     { "p", "Calculate expression: p EXPR", cmd_p },
9     { "w", "Set watchpoint: w EXPR", cmd_w },
10    { "d", "Delete watchpoint: d N", cmd_d },
11 };

```

6. 功能验证

在完成项目代码后，还需编译运行 NEMU 项目进行功能验证，测试每一项功能是否正常实现。

```

dev@ubuntu:~/ics2017/nemu$ make clean & make
[1] 2530
rm -rf ./build
+ CC src/monitor/debug/expr.c
+ CC src/monitor/debug/ui.c
+ CC src/monitor/debug/watchpoint.c
+ CC src/monitor/diff-test/gdb-host.c
+ CC src/monitor/diff-test/diff-test.c
+ CC src/monitor/diff-test/protocol.c
+ CC src/monitor/monitor.c
+ CC src/monitor/cpu-exec.c
+ CC src/memory/memory.c
+ CC src/misc/logo.c
+ CC src/device/keyboard.c
+ CC src/device/timer.c
+ CC src/device/vga.c
+ CC src/device/device.c
+ CC src/device/io/port-io.c
+ CC src/device/io/mmio.c
+ CC src/device/serial.c
+ CC src/cpu/exec/special.c
+ CC src/cpu/exec/data-mov.c
+ CC src/cpu/exec/control.c
+ CC src/cpu/exec/cc.c
+ CC src/cpu/exec/system.c
+ CC src/cpu/exec/logic.c
+ CC src/cpu/exec/exec.c
+ CC src/cpu/exec/prefix.c
+ CC src/cpu/exec/arith.c
+ CC src/cpu/intr.c
+ CC src/cpu/decode/decode.c
+ CC src/cpu/decode/modrm.c
+ CC src/cpu/reg.c
+ CC src/main.c
+ LD build/nemu
[1]+  Done                  make clean

```

图 1: 编译结果

编译正常，接下来运行项目，进行各项功能的测试：

```

问题  输出  调试控制台  终端  端口

dev@ubuntu:~/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 23:35:09, Apr 14 2026
For help, type "help"
(nemu) help
help - Display informations about all supported commands
c - Continue the execution of the program
q - Exit NEMU
si - Single step execution
info - Display register info (info r)
x - Examine memory: x N addr
p - Calculate expression: p EXPR
w - Set watchpoint: w EXPR
d - Delete watchpoint: d N
(nemu) si
100000:  b8 34 12 00 00          movl $0x1234,%eax
(nemu) si 3
100005:  b9 27 00 10 00          movl $0x100027,%ecx
10000a:  89 01                  movl %eax,(%ecx)
10000c:  66 c7 41 04 01 00      movw $0x1,0x4(%ecx)
(nemu) info r
eax: 0x00001234 ecx: 0x00100027 edx: 0x6abd31f3 ebx: 0x4d026fa6
esp: 0x49cd33d8 ebp: 0x07bc3bd8 esi: 0x5db763ad edi: 0x5d13ea23
eip: 0x00100012
(nemu) p 1+2*3
0x00000007 (7)
(nemu) p $eax+1
0x00001235 (4661)
(nemu) p (1+1)*10
0x00000014 (20)
(nemu) x 4 0x100000
Memory view at 0x00100000:
0x00100000: 0x001234b8
0x00100004: 0x0027b900
0x00100008: 0x01890010
0x0010000c: 0x0441c766
(nemu) w $eax
Watchpoint 0 created: $eax = 0x00001234
(nemu) info w
Num      Expr
0        $eax
(nemu) si
100012:  bb 02 00 00 00          movl $0x2,%ebx
(nemu) d 0
Watchpoint 0 deleted
(nemu) info w
No watchpoints.
(nemu) c
nemu: HIT GOOD TRAP at eip = 0x00100026

(nemu) q
dev@ubuntu:~/ics2017/nemu$ 

```

图 2: 测试结果

如图可知，各项命令功能正常，均符合实验要求与预期结果。

总体来看，NEMU 项目主要实现了作为计算机系统较为底层的功能模拟。从寄存器到运算再到监视点，除了需要灵活运用 c 语言这一底层语言之外，还需对于硬件的功能逻辑具有深入的理解；在将软件与硬件结合的过程中，逐渐掌握系统思维，进一步深入学习了系统的基础原理。

二、必答题

(1).i386 手册查阅

i.EFLAGS 寄存器中的 CF 位是什么意思？

CF 是进位标志 (Carry Flag)，是 EFLAGS 寄存器的第 0 位，用于反映无符号数运算的进位 / 借位状态。在加法运算时，若结果产生进位，则 $CF = 1$ ，否则 $CF = 0$ ；减法运算时，若结果产生借位，则 $CF = 1$ ，否则 $CF = 0$ 。

查阅范围：《Volume 1: Basic Architecture》 // Chapter 3: Program Execution Environment // Section 3.4.1: EFLAGS Register。

ii.ModR/M 字节是什么？

ModR/M 是 x86 指令中的寻址方式字节，位于操作码之后，用于指定操作数的寻址方式和寄存器内存位置。结构一共 8 位，分为 3 段分别对应不同的功能：Mod (2 位) 指定寻址模式、Reg (3 位) 指定寄存器编号、R/M (3 位) 指定有效地址的计算方式。

查阅范围：《Volume 2: Instruction Set Reference》 // Introduction // Instruction Format // ModR/M Byte Format。

iii.mov 指令的具体格式是怎么样的？

mov 目标操作数, 源操作数

其功能为在不改变源操作数的前提下，将源操作数的值复制到目标操作数中。

查阅范围：《Volume 2: Instruction Set Reference》 // MOV (Move Data/Byte/-Word/Doubleword/Quadword)

(2). 代码行数统计

在 PA1 开始前的提交处创建分支 PA0，作为新增代码的对照。接下来对两个分支分别进行代码量统计。这里使用 `find 7ics2017/nemu -name "*.c" -o -name "*.h" | xargs wc -l` 实现代码行数统计；为了防止空行影响代码量的计算，采用 `find 7ics2017/nemu -name "*.c" -o -name "*.h" | xargs wc -l | tail -n 1` 可以排除空行的影响。

在 MakeFile 中创建简便命令，添加如下代码：

```
1 # 统计总行数（含空行/注释）
2 count:
3     @find ./ -name "*.c" -o -name "*.h" | xargs wc -l | tail -n 1
4
5 # 统计纯代码行数（除空行外）
6 code-count:
```

```
7 @find ./ -name "*.c" -o -name "*.h" | xargs grep -v "^\\$\\$" |
   wc -l
```

使用以上方法，统计 PA1 分支新增的代码：

PA0:

```
1 dev@ubuntu:~/ics2017/nemu$ make count
2 3487 total
3 dev@ubuntu:~/ics2017/nemu$ make code-count
4 2817
```

计算得出，除去空行外，本次 PA1 相较于 PA0 一共新增 $3067 - 2817 = 250$ 行代码。

PA1:

```
1 dev@ubuntu:~/ics2017/nemu$ make count
2 3759 total
3 dev@ubuntu:~/ics2017/nemu$ make code-count
4 3067
```

(3).GCC 编译选项 -Wall 和 -Werror 作用与原因

-Wall: 开启 GCC 的所有警告信息。

-Werror: 将所有警告视为错误，导致编译失败，禁止警告代码被合并 / 发布。

使用较为严格的规范，可以有效保证代码质量。作为模拟计算机的底层实现，其严谨性尤为重要，让编译器将问题暴露出来并及时处理解决，能极大避免运行时的崩溃与漏洞等问题。同时，严格的规范标准也有利于我们养成良好的开发习惯，锻炼代码能力，从而更加适应贴近于工程级别的项目开发。